If Statements

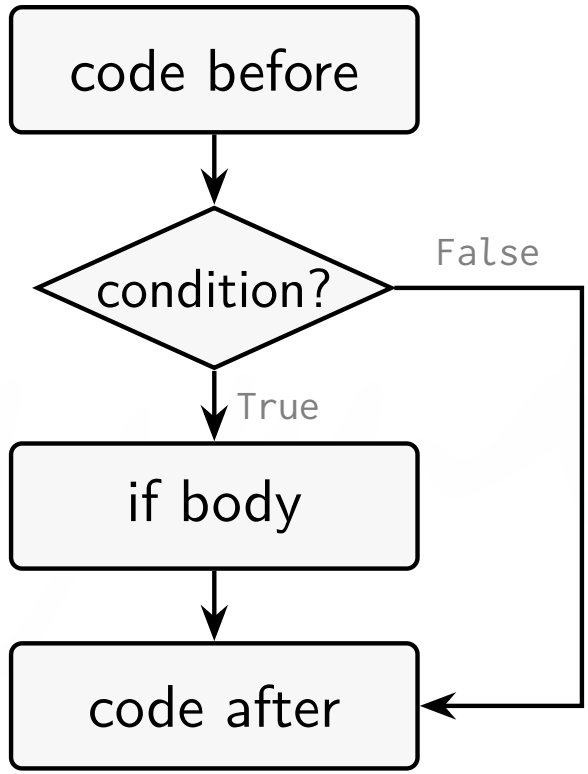
Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io



An **if statement** lets a program make a choice: run some code only when a condition is `True`.



An if statement starts with `if`, can have any number of `elif`s, and can end with one `else`. Each line ends with a colon and each body is indented.

```
if condition1:
    statements
elif condition2:
    statements
else:
    statements
```

```
time = 10
noon = 12
if time < noon:
    greeting = 'Good morning'
else:
    greeting = 'Good afternoon'
```

The conditions are checked top to bottom, and only the body of the *first* `True` one runs.

0 responses

What value does `x` refer to after this code runs?

```
x = 5
if x > 2:
    x = -3
if x > 2:
    x = 1
else:
    x = 3
```

A	-3	0%	0 votes
B	1	0%	0 votes
C	2	0%	0 votes
✓	3	0%	0 votes
E	5	0%	0 votes

0 responses

What value does `x` refer to after this code runs?

```
x = 0
if x < 5:
    x = 4
if x > 2:
    x = 1
else:
    x = 3
```

A	0	0%	0 votes
✓	1	0%	0 votes
C	3	0%	0 votes
D	4	0%	0 votes
E	5	0%	0 votes

Two separate `if`s are checked independently, so the first one can change the answer to the second.

```
x = 5
if x > 2:
    x = -3
if x > 2:
    x = 1
else:
    x = 3
```

line	condition	x
`x = 5`	—	5
`if x > 2`	True	-3
`if x > 2`	False	—
`else`	—	3

if/elif vs. if/if

0 responses

What value does `x` refer to after this code runs?

```
1  x = 5
2  if x == 5:
3      x = 6
4  elif x == 6:
5      x = 7
```

A	5	0%	0 votes
✓	6	0%	0 votes
C	7	0%	0 votes

0 responses

What value does `y` refer to after this code runs?

```
1  y = 5
2  if y == 5:
3      y = 6
4  if y == 6:
5      y = 7
```

A	5	0%	0 votes
B	6	0%	0 votes
✓	7	0%	0 votes

With `elif`, at most one branch runs. With two `if`s, both are checked.

```
x = 5
if x == 5:
    x = 6
elif x == 6:
    x = 7
```

`x` refers to `6`
the first branch ran, so the `elif` is skipped

```
y = 5
if y == 5:
    y = 6
if y == 6:
    y = 7
```

`y` refers to `7`
the second `if` is checked after `y` became `6`

grades.py

Program

```
 1  mark = 85
 2
 3  # with elif: only the first True branch runs
 4  if mark >= 80:
 5      letter = 'A'
 6  elif mark >= 70:
 7      letter = 'B'
 8  else:
 9      letter = 'C'
10
11  # with separate ifs: every condition is checked
12  if mark >= 80:
13      letter2 = 'A'
14  if mark >= 70:
15      letter2 = 'B'
```

Shell

```
=== RUN grades.py ===
```

if statements in functions

Both `if` and `return` control which code runs. A `return` exits the function immediately, so these two bodies do the same thing.

`return` in both branches

```
if num1 > num2:
    return num1
else:
    return num2
```

no `else` needed

```
if num1 > num2:
    return num1
return num2
```

Once a `return` runs, the function ends, so the code after the `if` only runs when the condition is `False`.

earlier_name.ipynb

Complete the following function.

In [ ]

```
def earlier_name(name1: str, name2: str) -> str:
    """Return the name, name1 or name2, that comes first alphabetically.

    >>> earlier_name('Jen', 'Paul')
    'Jen'
    >>> earlier_name('Colin', 'Colin')
    'Colin'
    """
    if name1 < name2:
        return name1

    return name2
```

In [2]

```
earlier_name('Jen', 'Paul')
```

```
'Jen'
```

ticket_price.ipynb

Complete the following function.

In [ ]

```
def ticket_price(age: int) -> float:
    """Return the ticket price for a person who is age years old.
    Seniors 65 and over pay 4.75, kids 12 and under pay 4.25 and
    everyone else pays 7.50.

    Precondition: age > 0

    >>> ticket_price(7)
    4.25
    >>> ticket_price(21)
    7.5
    >>> ticket_price(101)
    4.75
    """
    if age >= 65:
        return 4.75
    elif age <= 12:
        return 4.25
    else:
        return 7.50
```

In [3]

```
ticket_price(7)
```

```
4.25
```

0 responses

What should `ticket_price(65)` return?

4.75	0%	–	0
CORRECT ANSWER	GOT IT RIGHT · 0 OF 0	MOST COMMON	DISTINCT ANSWERS

format_name.ipynb

Complete the following function.

In [ ]

```python
def format_name(first: str, last: str) -> str:
    """Return the first and last names as a single string, in the form:
    last, first
    Mononymous persons (those with no last name) should have their name
    returned without a comma.

    >>> format_name('Cherilyn', 'Sarkisian')
    'Sarkisian, Cherilyn'
    >>> format_name('Cher', '')
    'Cher'
    """
    if last == '':
        return first
    else:
        return last + ', ' + first
```

In [3]

```
format_name('Cher', '')
```

```
'Cher'
```

In [4]

```
format_name('Chandra', 'Gummaluru')
```

```
'Gummaluru, Chandra'
```

functions that return bool

A condition is already a `bool`. When the branches just return `True` or `False`, **no if is required**: return the condition itself.

```
if value % 2 == 0:
    return True
else:
    return False
```

```
return value % 2 == 0
```

same_abs.ipynb

Complete the following function.

In [ ]

```python
def same_abs(num1: float, num2: float) -> bool:
    """Return True if and only if num1 and num2 have the same absolute value.

    >>> same_abs(-3.2, 3.2)
    True
    >>> same_abs(3.0, 3.5)
    False
    """
    return abs(num1) == abs(num2)
```

In [3]

```python
same_abs(-3.2, 3.2)
```

```
True
```

different_types.ipynb

Complete the following function.

In [ ]

```python
def different_types(obj1: object, obj2: object) -> bool:
    """Return True if and only if obj1 and obj2 are of different types.

    >>> different_types(3, '3')
    True
    >>> different_types(108.0, 3.14)
    False
    """
    return type(obj1) != type(obj2)
```

In [2]

```python
different_types(3, '3')
```

```
True
```

is_right_triangle.ipynb

Complete the following function. *(An extra exercise to try at home.)*

In [ ]

```
def is_right_triangle(side1: int, side2: int, hypotenuse: int) -> bool:
    """Return whether a triangle with sides of length side1, side2 and
    hypotenuse is a right triangle.

    >>> is_right_triangle(3, 4, 5)
    True
    >>> is_right_triangle(2, 2, 4)
    False
    """
    return side1**2 + side2**2 == hypotenuse**2
```

In [2]

```
is_right_triangle(3, 4, 5)
```

```
True
```

0 responses

For which ages does this function return `True`?

```python
def is_teenager(age: int) -> bool:
    if age < 13:
        return False
    else:
        if age > 18:
            return False
        else:
            return True
```

- ✓ 13 to 18, including both — 0% (0 votes)
- B 14 to 17 only — 0% (0 votes)
- C under 13 or over 18 — 0% (0 votes)
- D 13 and over — 0% (0 votes)

no_if_required.ipynb

Each of the following functions is correctly implemented, but is more complex than it needs to be. Replace each function body with a **single return statement**. You can use comparison operators `<`, `>`, `<=`, and so on, as well as boolean operators `and`, `or`, and `not`.

In [ ]

```
def can_vote(age: int) -> bool:
    """Return True if and only if age is legal voting age of at least 18 years.

    >>> can_vote(16)
    False
    >>> can_vote(21)
    True
    """

    """
    if age < 18:
        return False
    else:
        return True
    """
    return age >= 18
```

In [ ]

```python
def is_teenager(age: int) -> bool:
    """Return True if and only if age is a teenager between 13 and 18 inclusive.

    >>> is_teenager(4)
    False
    >>> is_teenager(16)
    True
    >>> is_teenager(19)
    False
    """

    """
    if age < 13:
        return False
    else:
        if age > 18:
            return False
        else:
            return True
    """
    return age >= 13 and age <= 18
```

A **constant** is a variable whose value should never change. By convention its name is in UPPERCASE, and it is defined at the top of the file.

```
TEEN_MIN = 13
TEEN_MAX = 18

def is_teenager(age: int) -> bool:
    """Return True if and only if age is a teenager between
    TEEN_MIN and TEEN_MAX inclusive.

    >>> is_teenager(16)
    True
    """
    return TEEN_MIN <= age <= TEEN_MAX
```

The name explains what the value means, and the value only needs to change in one place.

choosing docstring examples

Docstring examples are the first tests of a function, so choose them carefully.

(1) **follow the type contract and preconditions**

an example that breaks them doesn't tell us anything

(2) **show different behaviours**

for a `bool` function, one example that returns `True` and one that returns `False`

(3) **reach every branch**

for an if statement, an example for each of `if`, `elif` and `else`

tests one behaviour twice

```
>>> is_even(2)
True
>>> is_even(4)
True
```

tests both behaviours

```
>>> is_even(2)
True
>>> is_even(17)
False
```

0 responses

Recall `same_abs`. Which pair of function calls would be the best choice for docstring examples?

A (a) `same_abs(3.0, -3.0)` and `same_abs(5.5, 5.5)` 0% 0 votes

B (b) `same_abs(3.0, -3.0)` and `same_abs(-5.2, -5.2)` 0% 0 votes

✓ (c) `same_abs(3.0, -3.0)` and `same_abs(5.3, -10.9)` 0% 0 votes

get_longer.ipynb

Add two docstring examples to the function below that demonstrate *different* behaviours of the function.

In [ ]

```
def get_longer(s1: str, s2: str) -> str:
    """Return whichever of s1 and s2 is longer.

    Precondition: len(s1) != len(s2)

    >>> get_longer('cat', 'kitten')
    'kitten'
    >>> get_longer('kitten', 'cat')
    'kitten'

    """
    if len(s1) > len(s2):
        return s1
    else:
        return s2
```

In [2]

```
# Test your examples here
get_longer('cat', 'kitten')
```

```
'kitten'
```

In [3]

```
get_longer('kitten', 'cat')
```

```
'kitten'
```

0 responses

Assume we remove this precondition from `get_longer`. What might we want to change about the docstring examples?

```
1  # Precondition: len(s1) != len(s2)
```

- ✓ add an example where the strings have the same length — 0% (0 votes)
- B add an example with an empty string — 0% (0 votes)
- C add an example with two very long strings — 0% (0 votes)
- D nothing needs to change — 0% (0 votes)